Manabot: What We Have Demonstrated and the Recipes We Use

Jack Heart
Loopflow Studio

October 21, 2026

Written by AI. Not yet intended for distribution.

1 What we have demonstrated

Manabot trains agents to play a bounded, two-player subset of *Magic: The Gathering*. Batched self-play can produce policies that improve against fixed scripted opponents, batching sharply reduces training time, and several plausible interventions have failed to earn their cost. Evaluation bugs and weak opponents can make apparent progress misleading.

This section selects results from retained experiment reports; it is not a new experiment or a pooled meta-analysis. Each number belongs to its original world, code, recipe, seeds and opponents. October studies use the authored Allies/Lessons matchup; July search experiments use earlier decks and encodings. Current development is world w9, and current new-run policies use FiLM conditioning. Earlier measurements do not certify either change. Section 2 describes the present implementation separately.

1.1 Batching made useful experiments cheaper

The completed October 19 longer-learning study (protocol 017) trained 22 fits on RTX 5090 rentals, with two training seeds per recipe and 524,288 learner transitions per fit. Four raw checkpoints per fit were evaluated locally, totaling 18,304 replay-verified games. At equal collected experience, the width-64, two-layer model took 109–111 minutes with four streams, 12.6–13.3 minutes with 64 streams, and 4.8–5.2 minutes with 256 streams. Larger models showed no advantage at this horizon; two seeds are insufficient to rank their underlying performance.

The operational qualification rule required *both* seeds to score at least 50% against scripted greedy, at least 75% against random, and at least 20 percentage points above their own initialization against greedy, at a checkpoint and every later checkpoint. The 64-by-2 model at 256 streams qualified in a worst-seed 5.2 minutes; at 64 streams it qualified in 6.9 minutes. These are threshold crossings under a declared rule, not significance tests or estimates of eventual strength.

Batching changed learning as well as speed. At 64 streams, scores rose by roughly 131k–262k transitions and then flattened; at 256 streams they rose more slowly per transition and were still increasing at the endpoint. At equal experience, the comparison also changes Adam steps (8,192 versus 2,048), filtering and policy staleness. It therefore supports the whole recipe's cost advantage, not a causal claim that stream count alone improves learning. The deepest model at 64 streams used only about 57k of 131k planned selected exposures and performed poorly; its cause remains unexplained.

The settled GPU charge was $10. Local CPU evaluation occupied 7.6 coordinator hours, separate from that rental charge. Each endpoint had 104 games per anchor. Reported paired-deal intervals condition on one trained seed; they do not measure training variance or adjust for selection across many recipes. A cheap recipe that clears these anchors is useful infrastructure, but it is not yet evidence of broad strategic competence.

1.2 More optimization work did not reliably buy better play

Two completed October screens distinguish collection count from the work actually optimized. In the pooling and advantage-floor follow-up, three paired training seeds crossed masked-mean versus value-token aggregation with an absolute advantage floor of .01 versus zero. All twelve runs collected 153,600 learner transitions over 600 updates. Every endpoint was judged on the same 25 deals in four deck/seat assignments against scripted greedy; midpoint and endpoint together supplied 2,400 terminal, replay-verified games.

Removing the floor raised value-token optimizer exposures from 15,102–19,028 to 38,400, yet endpoint changes were $-1, +10, -12$ points across seeds. The mean was -1.00 points with a 95% paired seed/deal bootstrap interval of $[-11.33, 10.33]$. Masked mean's floor-removal effect was $+0.33$ points $[-4.00, 4.33]$. Neither intervention met the registered promotion or rejection rule. This is an unresolved result, not evidence of equivalence. Sparse comparisons at common elapsed cost also differ from equal-update comparisons. The value token changes shared policy representations, so this experiment does not isolate a critic-only mechanism.

The filter-scope mini screen kept the scalar masked-mean architecture, quantile/floor, learning rule and schedules fixed. Allowing the critic to train on every collected row, while filtering only actor rows, increased mean wall cost by 45% at 317,440 transitions. Final greedy-score changes were $+7, +7, -4$ points: mean $+3.33$, interval $[-4.33, 11.33]$, resampling paired seeds and shared deals. All 1,200 final/diagnostic and 2,400 monitoring games replayed exactly. The endpoint and sparse common-cost contrasts pointed in different directions. Extra critic coverage has not demonstrated that it earns its cost, and a lower loss on a different row population would not itself establish calibration.

A separate current-baseline debugging cohort failed its predeclared 10-point mean-gain criterion. Against a fixed random opponent, three short runs improved from 31.25% to 37.50%, from 33.33% to 41.67%, and from 33.33% to 39.58%. Frozen controls matched initialization. Mean gain was 6.94 points with a paired training-seed bootstrap interval of $[6.25, 8.33]$; all 432 games replayed. This short cohort cannot establish sustained self-play improvement.

1.3 Historical failures changed what we measure

The July draft treated extreme random-play seat effects as a domain finding. That interpretation was wrong. The reset-seeding investigation found that the Python wrapper accepted a seed without passing it to the engine. Repeated resets could replay one opening deal. Seat balancing alone did not repair that instrument: it balanced repeated rollouts of the same deal. The corrected record in correction ledger (F1–F3) reports deal-averaged random mirror parity (49.9% interactive, 49.8% standard; at least 1,000 games each) and retracts the proposed competence-emergent seat effect. We retain the old draft as history, not as current evidence.

The value-gate experiment makes the distinction concrete. Its learned value head saw 225 training games drawn from only three opening deals. After repairing evaluation seeding, search over that head beat greedy one-step use of the same head 60.25% $[55.4, 64.9]$ in 400 fresh-deal games. Yet value-search-64 lost 76.5% to random-rollout search-64 and 61.5% to random-rollout search-16

at approximate wall-time parity. Search helped relative to a weak greedy use of this particular estimator; it did not make that estimator an adequate replacement for full rollouts. The narrow training corpus and single training seed prevent a general verdict on learned value functions.

Other failures remain useful warnings with bounded scope. The July expert-iteration attempt produced a student that lost 74–26 to its predecessor in 100 head-to-head games. Its affordable new teacher used eight simulations and short policy continuations; this refutes that attempted crank, not expert iteration in principle. Its historical judge and data limitations remain in the report. The control-competency suite showed that a search pilot with attractive aggregate wins could fail known-correct control lines. Win rate against random is not a sufficient behavioral test.

What remains unestablished. Monitoring curves, toy learnability controls, legal replay and runnable belief/search pipelines answer different questions. None supplies a held-out human comparison, unseen-card transfer result, exploitability bound, or broad method ranking, including an advantage over PPO. Retained reports preserve each cohort's scope and acceptance rule, including failures.

2 Methods: recipes and their influences

This section assumes familiarity with RL and describes implementable choices rather than the general difficulty of card games. Code paths refer to repository base `a04dcdc2`. Experiments above retain their own source identities. The recipes combine established methods with local engineering choices; they are not proposed as new algorithms.

2.1 Keep rules, observations and experiment identity together

Recipe. Use managym as the authoritative Rust rules engine and manabot as its learning client. Construct the authored deck and sideboard setup, encode only the acting viewer's observation, and score the engine's legal offers. Bind checkpoints to native world version, rules schemas, both deck/sideboard pairs, content manifest and input schema. Reject mismatches before loading weights. A rules-meaning or observation/action-shape change starts a new world; never draw a single learning curve across incompatible worlds.

Implementation. `manabot/env/` owns wrappers and encoding; `manabot/model/world.py` owns checkpoint bindings. Academy is the inspection/play client and does not define the training objective. Etude is the separate experiment launching/recording system. The deterministic, forkable engine makes simulated branches practical, but a sampled hidden world is not an observation that a policy may inspect. Source-pinned replay and viewer-equivalence checks are local integrity requirements, not borrowed claims of strategic correctness.

2.2 Score legal offers from objects and state

Recipe. Embed visible players, cards and permanents; apply masked object attention with perspective/ownership information; gather each legal action's type/features and its referenced object embeddings; produce one logit per offer and mask illegal padding. The compiled-match path also embeds checked semantic card programs. These joins distinguish card definitions, but their existence is not evidence of zero-shot transfer.

The attention block follows the multi-head attention/residual/normalization pattern of Vaswani et al. [2017]. It is a small object encoder, not the original sequence-to-sequence Transformer. New-run policies additionally use a scale-only adaptation of FiLM [Perez et al., 2018]. With encoded objects h_j, validity m_j, and informed action embedding e_a:

$$s = \frac{\sum_j m_j h_j}{\max(1, \sum_j m_j)}, \qquad \widetilde{e}_a = e_a \odot (1 + g(s)).$$

A shared head scores $\widetilde{e}_a$. Unlike a state-only additive offset shared by every logit, this multiplication can change relative action preferences. There is no separate FiLM shift term here. The masked mean and shared action head preserve object/offer permutation contracts.

Implementation. `Agent` in `manabot/model/agent.py` owns object attention, focus gathering, FiLM and the critic readout. The current ordinary baseline uses a width-64, two-layer, four-head semantic model with a scalar masked-mean critic and `policy_film=True`. Omitted FiLM settings retain historical additive-policy semantics. Value-token and categorical-outcome models are explicit alternatives; do not relabel a scalar checkpoint as categorical or attribute earlier additive-policy results to FiLM.

2.3 Collect transitions between decisions of the same player

Recipe. Batch independent games. Within a collection batch freeze the policy, but allow games to continue across successive updates. A learner row ends at that player's next decision or at game termination; intervening opponent actions receive no learner policy gradient. Save chosen actions, full legal behavior log probabilities, values, terminal outcomes and the exact paused next observation. Bootstrap from that observation; end markers stop the trace before reset observations enter it. Treat truncation as failure, not as a draw.

Influences and implementation. The PPO control uses Schulman et al. [2017] and GAE [Schulman et al., 2016]. `manabot/model/train.py` explicitly follows CleanRL's flat rollout-tensor organization [CleanRL contributors, 2026]; this is an implementation influence, not a claim of byte-for-byte derivation. `SeatRoutedCollector` and `transition_gae` in `manabot/sim/net_opponent.py` supply the same-viewer collection path. For transition-end flag d_t:

$$\delta_t = r_t + \gamma(1 - d_t)V_{t+1} - V_t, \qquad A_t = \delta_t + \gamma\lambda(1 - d_t)A_{t+1}.$$

Targets and saved behavior estimates are detached. Terminal signed outcomes label training rows without exposing true hidden cards as policy inputs. Games spanning updates make this close-to-on-policy continuation; there is no trajectory-level correction for changing historical state occupancy.

2.4 Use the damped move update as an explicit adaptation

Influence. The ordinary self-play recipe adapts Ataraxos's move learning, specifically supplement S3.4, equations (5)–(6), and Table S7 [Sokota et al., 2026], including its clipped likelihood-ratio surrogate. It does not implement the full Stratego system or DeepNash's R-NaD.

Recipe. With current policy π, saved collection policy μ, detached advantage A, reference ρ and selected-action ratio $r = \pi(a)/\mu(a)$, minimize the selected-row mean of

$$-\min\{rA, \mathrm{clip}(r, 1 - \epsilon, 1 + \epsilon)A\} + c\, D_{\mathrm{KL}}(\pi\|\mu) + \tau\, D_{\mathrm{KL}}(\pi\|\rho).$$

Both reverse KLs sum over the entire legal support. The first damps movement from collection behavior; the second pulls toward a fixed reference. Our default reference chooses uniformly among present action types, then uniformly among offers of the chosen type. This replaces Stratego's piece/move grouping with an MTG engineering choice; a spell with many targets does not receive proportionally more prior mass solely from offer count.

Use $\gamma = 1$, policy trace $\lambda = .5$, and value trace $\lambda = .8$. Keep rows whose absolute advantage reaches the inclusive .75 quantile and the .01 floor; ties can retain more than one quarter. Do not normalize advantages after selection. Default filtering applies to actor and critic; the actor-only switch keeps all critic rows. Perform one epoch in increasing learner-timestep groups across streams, without shuffling. Earlier groups change parameters, so saved action likelihoods still matter within that epoch.

For categorical loss/draw/win heads, terminal targets are one-hot outcomes and nonterminal targets recursively mix next predictions and lambda returns; minimize soft-target cross-entropy. The ordinary baseline instead uses a scalar critic and MSE lambda-return loss, an explicit representation departure from Ataraxos. The value loss has unit weight. Gradients reach current policy/critic heads and their shared encoder; behavior distributions, references and targets remain fixed.

Implementation and constants. `manabot/training/ataraxos.py` owns loss/targets/update grouping; `references.py` owns the reference; `selection.py` owns selection. `AtaraxosMoveLearning`

in `models.py` sets $\epsilon = .2$, $c = .1$, gradient norm cap .267, and

$$\eta_i = \text{clip}(.5/i^{1.1}, 5\times10^{-6}, 10^{-4}), \qquad \tau_i = .05/i^{.3}.$$

Adam uses the existing betas .9/.999 and epsilon 10^{-5}. Evaluation EMA uses decay .999, updated once per collection/update iteration, including empty filters; it is not a fictitious-play mixture. The source leaves some details unspecified (including Adam constants, quantile interpolation and EMA initialization); `docs/ataraxos.md` records our choices rather than claiming exact reproduction.

2.5 Freeze the whole recipe, including its clock

Recipe. Declare model, world/setup, learner, schedules, opponent mode, collection budget, checkpoint selection and evaluation before launch. Preserve raw and EMA exports separately. Collection updates, learner transitions, selected exposures and elapsed cost are distinct quantities. Retain empty filters and failed attempts in that accounting.

Implementation. `manabot/training/baseline.py` resolves the current `baseline-v2`: four streams by 64 transitions, 128 updates by default, current-self collection, terminal outcome reward and the FiLM scalar model above. Its horizon-dependent schedule maps first and last positions onto historical iterations 1 and 10,000; a shorter run is a compressed schedule, not a prefix. The frozen GPU regime `experiments/regimes/batched-baseline.json` retains its own tested settings and identity. Neither recipe silently inherits the other's evidence.

PPO remains a selectable control, with its own discounting, normalization and reuse settings. Do not call a bundled PPO-versus-Ataraxos recipe comparison a single-mechanism ablation. `TrainingRegime`, the experiment runner and VerifyStore retain resolved configuration, source/native/setup identities, seed lineage, exports and cost. A bounded local run checks the operational path; it does not certify the scientific comparison.

2.6 Separate search labels from strategic evidence

Recipe. A frozen-policy local teacher samples compatible hidden worlds, evaluates every positive-support root action with frozen-policy continuations, and aggregates action values before choosing a distribution. Every simulated actor uses its own ordinary observation. Compatible-prior sampling and a learned posterior are different declared choices. Exact-history likelihood support is restricted; unsupported events fail rather than becoming fabricated posterior evidence.

The local policy update follows Ataraxos supplement S3.7, equations (7)–(8). For base policy p, reference ρ, and sampled values $\widehat{Q}$, it maximizes

$$\sum_a q_a \widehat{Q}_a - \alpha D_{\text{KL}}(q\|\rho) - \beta D_{\text{KL}}(q\|p), \quad q_a \propto \exp\left(\frac{\widehat{Q}_a + \alpha\log\rho_a + \beta\log p_a}{\alpha+\beta}\right).$$

`manabot/sim/local_update.py` owns the teacher and receipts. Distillation uses fixed soft targets on retained roots with policy cross-entropy; gradients do not enter the teacher. Whole-game partitions separate training and validation. The critic head is not fitted by these policy-only stages, though the shared encoder can drift. This is a later local-update path, distinct from the July flat-search experiment. It is runnable machinery whose strategic benefit still needs a matched comparison.

2.7 Measure outcomes, behavior and cost separately

Recipe. Freeze an opponent cohort, independent training seeds, held-out paired deals, deck/seat assignments and inference envelope. Score complete games and replay every retained command.

Keep inspected monitoring deals separate from a final holdout. Bootstrap whole paired-deal blocks for a fixed checkpoint; resample training seeds as well when estimating a method contrast. A large number of games from one seed cannot supply independent training replicates. Keep payoff matrices and competency failures alongside aggregate scores.

Implementation. `manabot/arena/` owns registration, matches, replay and seat-aware ratings; checkpoint monitoring lives in `monitor_evaluation.py` under `manabot/training/`. Ratings are local to a world, matchup suite, information boundary, compute class and arena version. `manabot/verify/learnability/` checks capacity/invariance, while its full diagnostic includes reward-driven PPO and reload. Passing that diagnostic is a software/optimization result, separate from full-game strength. Frozen experiment reports, not live process state or this paper's prose, remain the authority for measured results.

References

CleanRL contributors. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. https://github.com/vwxyzjn/cleanrl, 2026. Codebase; accessed October 21, 2026.

Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1709.07871.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Samuel Sokota, Eugene Vinitsky, Hengyuan Hu, Zhiyuan Fan, J. Zico Kolter, and Gabriele Farina. Scalable decision-making for games of imperfect information. *Nature*, 658:55–59, 2026. doi: 10.1038/s41586-026-11036-y. URL https://www.nature.com/articles/s41586-026-11036-y. Move learning: supplementary S3.4, equations (5)–(6), Table S7; local search update: S3.7, equations (7)–(8).

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.